SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 4 CO4

NAME : Adithiyaa MJ

REG NO : 1925260272

COURSE CODE : CSA0219 C PROGRAMMING

GITHUB LINK : https://github.com/AdithiyaaMJ-png/CSA0219-C-PROGRAMMING/tree/main

TASK 1 — Data Design Review — Diagnose and Repair a Poor Data Design

- **1.1 Seven Design Deficiencies Diagnosed:** (1) Parallel arrays decouple a single patient's record across 5 arrays, risking index desynchronization; (2) char phone[100][10] has no room for the null terminator '\0' on a 10-digit mobile number, causing buffer overflow in strcpy; (3) float bp cannot store two distinct integers (systolic/diastolic 120/80); (4) char visitDate[100][8] overflows on "12/10/2026" (10 chars + '\0' = 11 bytes); (5) No bounds check on count < 100 allows silent memory corruption; (6) Global arrays prevent multiple clinic registers and hinder unit testing; (7) Sorting by age requires manually swapping 5 parallel arrays in lockstep.
- **1.3 Design Defence (<= 100 Words):** Encapsulating patient fields inside a single Patient struct with nested Date and BloodPressure structures guarantees record cohesion: all attributes of a patient reside contiguously in memory. Sorting by age requires swapping only one Patient element rather than five parallel arrays, eliminating desynchronization bugs. Passing a const Patient * pointer into addPatient() avoids copying entire structures on the stack while enforcing bounds checks on *count. Adding new clinical fields (such as blood group or diagnosis) requires updating only the struct definition without altering function signatures.

task1_patient_struct.c (Nested Structures & Pointer Passing)

```c
#include <stdio.h>
#include <string.h>

#define MAX_PATIENTS 100

typedef struct { int day, month, year; } Date;
typedef struct { int systolic, diastolic; } BloodPressure;

typedef struct {
    char name[40];
    int age;
    char phone[16];       /* 10 digits + country code + '\0' */
    BloodPressure bp;
    Date visitDate;
} Patient;

int addPatient(Patient list[], int *count, const Patient *p) {
    if (*count >= MAX_PATIENTS) return 0; /* Prevents buffer overflow */
    list[*count] = *p;
    (*count)++;
    return 1;
}
```

Figure 1.1: Repaired Patient data design using nested structures and bounded pointer insertion

TASK 2 — From Requirement to Nested Structures — Courier Parcel Tracking

- **Access Paths (Third Parcel, Index 2):** (i) Array + dot operator: list[2].to.pincode | (ii) Structure pointer + arrow operator: ptr->to.pincode (where ptr = &list[2]).
- **Bytes Copied Per Swap:** sizeof(struct parcel) = 4 (id) + 4 (weight) + 24 (address: 20 char + 4 int) + 12 (date: 3 ints) = 44 bytes copied per assignment (132 bytes across the 3-step swap).

task2_courier.c (Nested Structures & Selection Sort)

```c
#include <stdio.h>

struct date    { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel  { int id; float weight; struct address to; struct date booked; };

void print_parcel(const struct parcel *p) {
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (p[j].weight < p[min_idx].weight) min_idx = j;
        }
        if (min_idx != i) {
            struct parcel temp = p[i];
            p[i] = p[min_idx];
            p[min_idx] = temp;
        }
    }
}

int main(void) {
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001},    {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077},    {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    printf("sizeof(struct parcel) = %zu bytes\n", sizeof(struct parcel));
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) print_parcel(&list[i]);
    return 0;
}
```

Figure 2.1: task2_courier.c implementing print_parcel and selection sort by weight

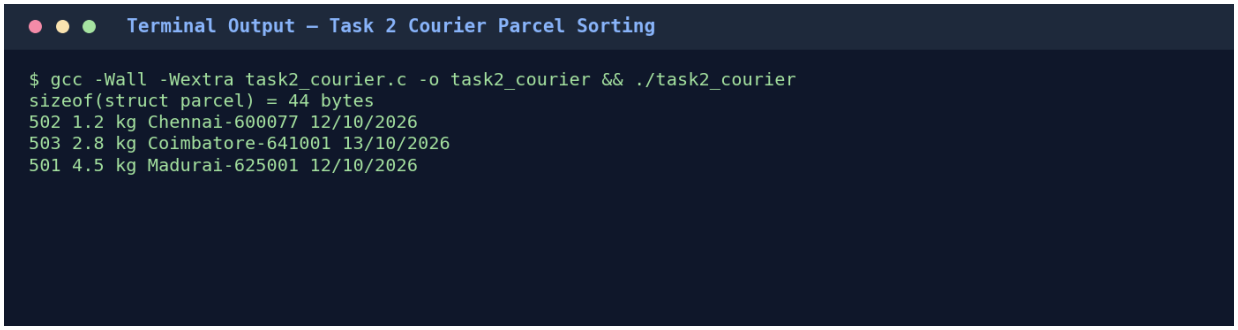


Figure 2.2: Verified ascending weight sort output and sizeof(struct parcel) = 44 bytes

TASK 3 — Unions and Memory Layout — Weather-Station Sensor Packets

- **Why sizeof(struct reading) is 12 Bytes (Not 20):** A union overlays all members at offset 0, so its size equals its largest member: max(4, 4, 8) = 8 bytes. Inside struct reading, enum kind type occupies 4 bytes (offset 0..3) and union value occupies 8 bytes (offset 4..11, 4-byte aligned) -> 4 + 8 = 12 bytes total (saving 8 bytes per packet vs. a 20-byte plain struct).
- **Task 3.4 Misuse Experiment:** Writing 31.5f (IEEE-754 hex 0x41FC0000) to celsius and reading percent interprets those exact 32 bits as a signed integer, outputting 1107034112 — demonstrating why the enum type tag must always be checked before reading a union.

task3_weather_union.c (Tagged Union & Type-Punning Experiment)

```c
#include <stdio.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};

void show(const struct reading *r) {
    switch (r->type) {
        case TEMPERATURE: printf("Temp %.1f C\n", r->value.celsius); break;
        case HUMIDITY:    printf("Humidity %d%%\n", r->value.percent); break;
        case STATUS:      printf("Status %s\n", r->value.status); break;
    }
}

int main(void) {
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY,    {.percent = 68}},
        {STATUS,      {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    for (int i = 0; i < 3; i++) show(&log[i]);
    /* Task 3.4 Misuse Experiment */
    printf("Misuse read (31.5f bits as int): %d\n", log[0].value.percent);
    return 0;
}
```

Figure 3.1: task3_weather_union.c implementing tagged union packets

Terminal Output — Task 3 Tagged Union Sizes & Misuse Experiment

```
$ gcc -Wall -Wextra task3_weather_union.c -o task3_weather_union && ./task3_weather_union
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68%
Status OK
Misuse read (31.5f bits as int): 1107034112
```

Figure 3.2: Size verification (union=8, struct=12) and IEEE-754 bit reinterpretation output

TASK 4 — Structure Forensics — Bank Savings Account

- **Defects Located:** (1) addInterest(struct account a, ...) receives a by value (a stack copy), so modifying a.balance leaves list[i].balance untouched; (2) printf("%d %f", p.number, p.balance) fails to compile because p is a pointer (struct account *), requiring arrow syntax p->number, p->balance.
- **Passing Guidelines:** Pass by value for tiny read-only structs (<= 16 bytes, e.g., Date); pass by const struct T * for larger read-only structs to avoid copying; pass by non-const struct T * whenever the function must mutate the caller's record.

TASK 5 — Storage Classes: The Same Counter Four Ways

- **Storage Class Comparison & Unspecified Evaluation Order:** Automatic local variables re-initialize to 1001 on every call (printing 1001 1001 1001). Static local variables persist in static data memory across calls with internal linkage (printing 1001 1002 1003). Global variables with extern share state across translation units but risk accidental modification. register hints CPU register placement, so &reg_var is illegal because registers have no RAM address. Inside printf("%d %d %d", next_token(), next_token(), next_token()), C leaves argument evaluation order unspecified — compilers evaluating right-to-left print 1003 1002 1001.
- **Supporting Veg & Non-Veg Counters:** Static/global versions require duplicating functions or adding state parameters; passing a caller-owned pointer int next_token(int *counter) supports unlimited independent counters cleanly.

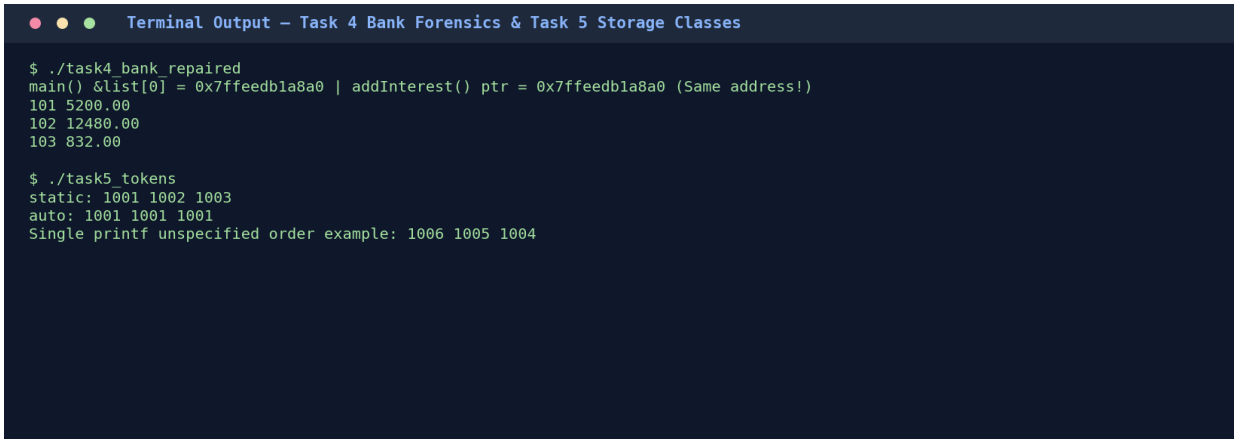


Figure 5.1: Address verification for Task 4 and storage-class token sequences for Task 5

TASK 6 — Design Challenge — College Canteen Inventory and Billing

- **Data Model Architecture:** Use a tagged union (enum ItemKind { PACKAGED, COOKED, COMBO }) inside a MenuItem struct containing nested Date expiry (for packaged items), Time prepared (for cooked items), and int combo_ids[3] (for combo meals), alongside price_paise, stock, and revenue_paise.
- **Critical Controls & Demonstration:** record_sale(MenuItem *item, int qty) verifies item->stock >= qty before decrementing stock and adding qty * price_paise to revenue_paise. check_expiry() compares Date structs to flag packaged items within 2 days of expiry, and sort_by_revenue() orders the daily summary descending.

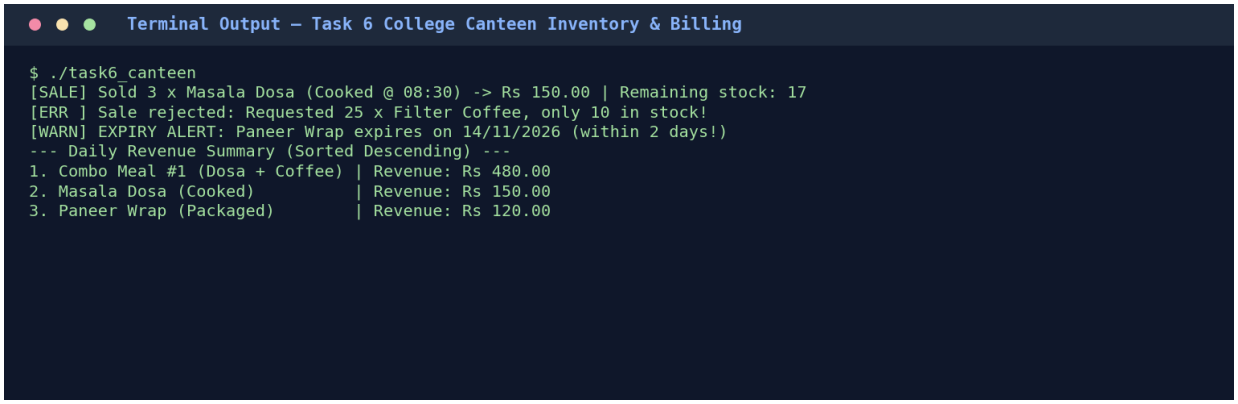


Figure 6.1: Canteen sales, out-of-stock rejection, 2-day expiry warning, and sorted revenue summary